

Mesh-to-CAD AI

Modeling Seminar

at

University of Bayreuth

Chair of
Scientific Computing
in cooperation with

Rafinex and Chair of Design Theory and CAD

By: Anke Neumüller, Antony Youssef
Supervised by Prof. Dr. Tobias Rosnitschek, Sandra Aziz
6 January 2025



Contents

1 Introduction **1**
1.1 Problem statement 2
1.2 Aim 2

2 Methodology and literature review **3**
2.1 Methodology 4
2.2 Literature review 7
2.2.1 Constructing 3D geometry based on 2D images 7
2.2.2 AI tools used to generate CAD models 9

3 Implementation **13**
3.1 STL format to matrix of photos 13
3.2 Matrix of photos to text description 14
3.2.1 Qwen 2.5-Model (VLM) 14
3.2.2 Data Preparation 14
3.2.3 Low-Rank Adaptation of language models 16
3.2.4 Fine-tuning of the Qwen VLM 18
3.3 Text description to CAD sequence 23
3.4 CAD sequence to parametric CAD model 24
3.4.1 Methodology 24
3.4.2 Implementation 25
3.4.3 Results 26
3.5 Optimization process 27
3.5.1 Preprocessing 27
3.5.2 Definition of cost function 27
3.5.3 Constraint enforcement 28
3.5.4 Optimization method 29

4 Results **32**

5 Conclusion and future work **36**
5.1 Future work 36

Chapter 1

Introduction

In modern engineering, topology optimization has become an essential tool for designing efficient lightweight structures. While preserving structural performance, unnecessary material is removed, as shown in Figure 1.1. Results are often geometries that would be difficult to design manually. They are usually exported in mesh-based formats such as STL.

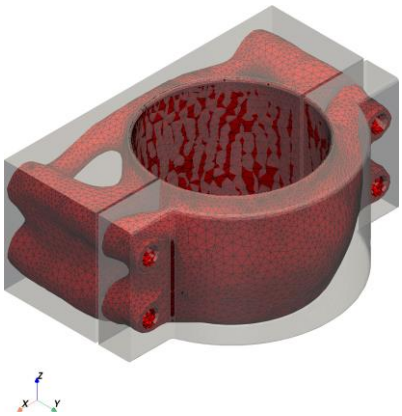


Figure 1.1: Motor bracket before and after topology optimization.[1]

In STL format the boundary of the geometry is represented by a surface of triangles, where each triangle is described by its outward normal and the coordinates of its vertices. [15]

For visualization and additive manufacturing, this is sufficient.

[1]This topology optimized CAD model for motor bracket has been provided by Rafinex

When designing CAD parts from scratch, the geometry is created successively. Various geometries are combined, where geometries are created using 2D sketches, which are then transformed into 3D shape using 3D operations such as extrusion. This workflow allows parametrization. When changes need to be implemented, such as a different diameter or an adjusted edge length, this can be done easily by adjusting parameters. STL format does not provide this parametric information which allows for modifications. This leads to the problem addressed in this study.

1.1 Problem statement

Since STL files are not editable, they restrict applications like design iterations, tolerance adjustments and integration into larger systems. In industrial applications, changes in geometry are often inevitable. Converting mesh-based results into parametric CAD models is a challenge for engineering design workflows. Until now, engineers had to reconstruct geometry manually by reverse engineering to obtain parametric models. This process is time- and cost-intensive. Therefore, an automated method to get editable parametric CAD models from STL meshes is highly desirable.

1.2 Aim

The aim of this project is to develop a method that uses AI tools to convert STL meshes into parametric CAD models. This is done by combining large language models and optimization techniques. The goal of this project is to automate the process for converting STL files into parametric CAD files to obtain a result which is more useful for the industry. In the following, the approach to this project will be outlined.

Chapter 2

Methodology and literature review

To find a suitable approach, the requirements are first defined. The generated model has to provide a parametric design and it should be able to generalize. So the method has to be able to include parametric information and generate reasonable results even if the geometry differs from the initial training dataset.

Following the literature review, large language models (LLMs) appeared most promising. LLMs are a type of AI system using deep learning, neural networks and transformer architecture which allows the model to interpret many tokens simultaneously and to capture long-range dependencies effectively. The model is trained to learn statistical patterns and contextual relationships in language. After extensive pretraining, LLMs can be fine-tuned on smaller, task-specific datasets to optimize the model for given tasks. This allows the model to be optimized with few additional data. LLMs are especially useful for repetitive tasks, generalizing from existing data and transfer learning. [1]

Since all requirements can be potentially fulfilled by LLMs, they will be used for this project. Various LLMs for converting meshes into CAD geometry already exist. A method which has the potential to generate parametric CAD geometries is searched. That will be discussed in the next section.

2.1 Methodology

As the aim of this project is based on generating parametric CAD models, large language models (LLMs) and vision language models (VLMs) have been proposed as a means for developing an approach/tool that could generate a parametric CAD model from data format like STL-mesh format. Several works have relied on large language models (LLMs) and vision language models to generate CAD models from different forms of input, as shown in Table 2.1. A brief literature review for the AI tools which are used in generating CAD models will be provided in a later section of this Chapter.

Table 2.1: Comparison of AI based methods to generate 3D CAD models [7].

Method	Input	Output	Model Architecture
Text2CAD	Text	3D	GPT-3.5 based
GenCAD	Image	3D	Transformer encoder/decoder + Contrastive image encoder + Diffusion prior
IdeaToCAD	Image + Text	3D	GPT-4o
CAD-Coder	Image + Text	3D	LLaVA 1.5

As shown in Table 2.1, these tools always require another technique which should convert the STL-mesh into either text or images, or another data format. Given the success of several approaches which have used 2D photos to create 3D CAD models [12], [16], [8] – these works will be reviewed in the next section of this chapter – this step has been adopted to generate a matrix of 2D photos for STL-mesh so the AI tools which have been developed to generate CAD models could be used.

One of the proposed AI tools for generating parametric CAD is Text2CAD [10]. As it will be reviewed later in this chapter, compared to other tools, this has been developed mainly to generate these parametric CAD models. This tool will be further modified, as will be clarified in the next chapter.

Although this tool provides a perfect match for the objectives of this project, the input for it requires a detailed descriptive non-structured sequence based

on the structured CAD sequence of the required CAD model. This non-structured CAD sequence is a set of instructions which should be followed to construct the given CAD model; however, as it is non-structured, using CAD codes to work on this non-structured frame to get CAD models from them is not possible. A comparison between the structured and non-structured CAD sequences is shown in Figure 2.1 and Figure 2.2, which is for constructing a hollow cylinder.

As shown in Figure 2.1, this is for constructing a hollow cylinder. As writing such a non-structured CAD sequence is challenging and requires an expert level of writing, a VLM tool, "Qwen2.5-VL" [3], will be fine-tuned on the data generated by the Text2CAD research work to produce such a non-structured CAD sequence for the matrix of 2D photos of the CAD model.

The generated CAD sequence has to be transformed into CAD geometry. For this a Python script which converts the CAD sequence into a Python script for FreeCAD is generated.

Since the previous steps did not include any information about dimensions of the STL file an optimization step has to be performed. This step ensures that the dimensions scale in right portions and the output model matches the input model.

The full proposed implementation is shown in Figure 2.3; the major stages for this plan are:

- Generate a 3×3 matrix of photos for CAD model views using a FreeCAD macro
- Fine-tune and test Qwen VL model to output text prompt (non-structured CAD sequence) for a given matrix of photos for CAD model
- Install and modify Text2CAD to output the structured CAD sequence for the given text prompt
- Write a Python script to transform and modify the CAD sequence into macro.py for FreeCAD
- Incorporate an optimization algorithm in the Python code to ensure the final CAD model matches the input model

As the proposal of transforming an STL-mesh from a 3D model to a matrix of 2D photos, and also the AI tool used to generate the CAD model, are major steps and ideas in this project, a literature review will discuss both in the coming sections. The other stages will be reviewed and discussed in their corresponding chapters.

```
Structured CAD Sequence:
- Sketch:
   - CoordinateSystem:
       - Rotation Matrix [1.0, -0.0, 0.0, 0.0, 2.220446049250313e-16, 1.0, -0.0, -1.0,
                          2.220446049250313e-16],
       - Translation [0.   0.75 0.  ]
   - Face:
      - Loop: Start Point: [0.0018, 0.0018], Direction: counterclockwise
          - Circle: center([0.2353 0.2353]), radius(0.2335), pt1 [0.23529412 0.46875   ]
      - Loop: Start Point: [0.1103, 0.1103], Direction: counterclockwise
          - Circle: center([0.2353 0.2353]), radius(0.125), pt1 [0.23529412 0.36029412]

- ExtrudeSequence: (extent_one: 0.75, extent_two: 0.0, boolean: 0,
                    sketch_size: 0.46875) Euler Angles [ 0. 0. -1.57079633]
```

Figure 2.1: A structured CAD sequence to construct a hollow cylinder.

.. a **cylindrical object with a hollow center.**

.. **Euler angles (0.0, 0.0, -90.0)** and a **translation vector of (0.0, 0.75, 0.0).**

.. Loop 1 is a **circle** with a **center** at **(0.192, 0.192)** and a **radius of 0.192**.

.. **Loop 2** is also a **circle** same center at **(0.192, 0.192)** with **radius of 0.12.**

.. **sketch scale** parameter of **0.384** and transform it into a 3D sketch.

Figure 2.2: A non-structured CAD sequence to construct a hollow cylinder.

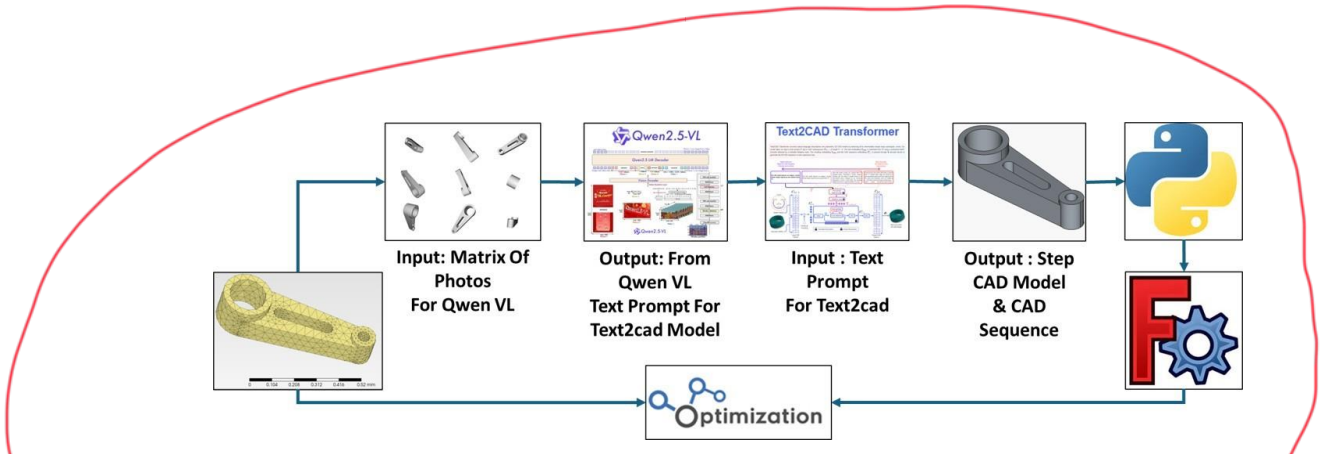


Figure 2.3: Proposed implementation: AI-based conversion from mesh to CAD.

At the beginning

+ Excel Macro!

2.2 Literature review

2.2.1 Constructing 3D geometry based on 2D images

The proposed approach has been - as previously described and clarified - based on converting the STL-mesh format from 3D model into matrix of photos for different 2D views of the CAD model. This matrix of 2D photos will be used through a chain of process to construct the 3D parametric CAD model, as previously shown in Figure 2.3.

In this context, several research work have been developed to convert 2D photos into 3D models; in [8], a convolutional neural network framework has been used to predict 3D particles shapes based on 2D images. That approach which is shown in Figure 2.4 have based on the following stages :

- Database Generation: This phase includes pre-processing tasks such as creating 3D models, generating 2D images from multiple viewpoints, and converting the 3D mesh (.STL) files into voxel models. These voxel models serve as the ground truth for training and evaluating the AI model.
- AI Model Construction and Training: In this phase, a Convolutional Neural Network (CNN) is developed and trained. The CNN takes multiple 2D images as input and predicts a 3D voxel grid representing the object. A post-processing step then converts the predicted voxel grid into a 3D mesh using marching cubes and Laplacian smoothing.
- 3D Particle Models and Shape Parameter Extraction: This step involves generating 3D particle models from the predicted voxel grids and extracting shape parameters, such as sphericity, elongation, aspect ratio, and flatness, from the point clouds. Additional post-processing is performed to compare the predicted 3D particle models with the ground truth 3D meshes generated in the first step.

Similarly, in [16], an approach based on using Convolutional Neural Networks to reconstruct the 3D morphology of irregular sand particles from 2D images has been proposed, where over 100,000 particles were collected from three different sources: naturally occurring particles (desert sand), synthetic particles (lunar soil simulant), and digitally generated particles. A deep learning method, utilizing voxel-based representations of particle morphology combined with multi-dimensional convolutional neural networks, was employed to efficiently reconstruct particle shapes, as shown in Figure 2.5.

In [12], an approach based on training the deep learning EfficientNetB0 model

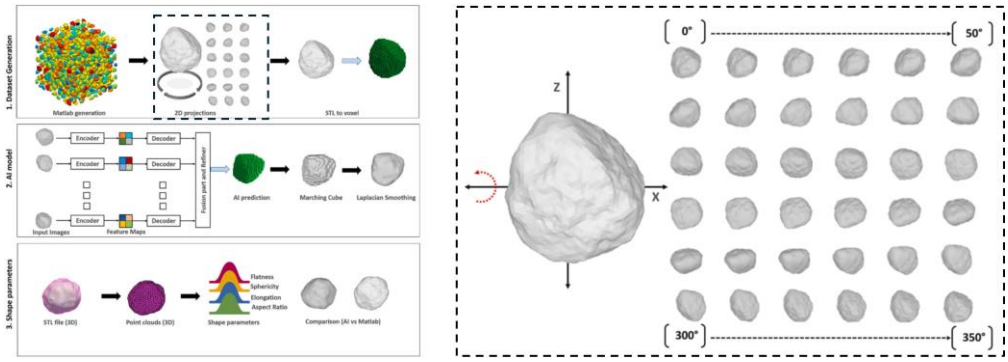


Figure 2.4: 3D-CNN methodology for generating realistic 3D particle models from 2D projections [8].

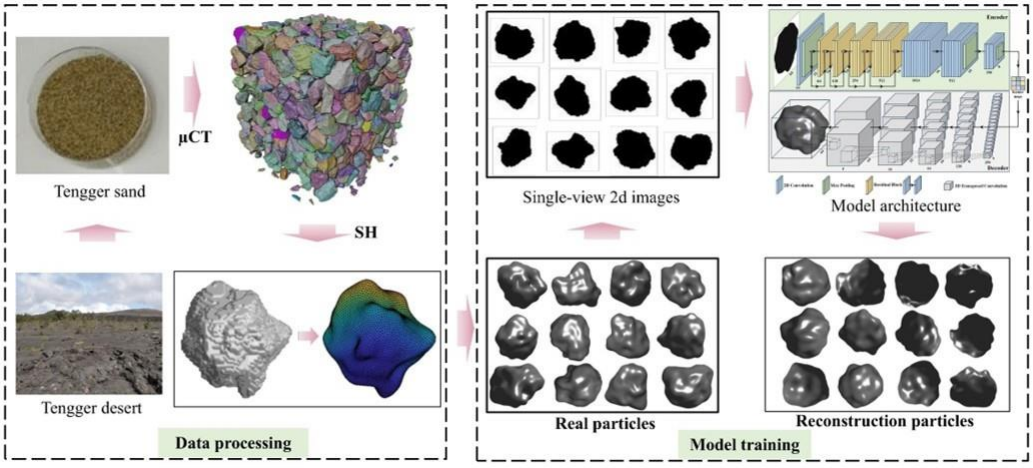


Figure 2.5: Pixels-to-voxels of particles (PVP) model based on the Pix2Vox has been proposed to reconstructing the 3D morphology of particles which are represented by 2D images [16].

for constructing 3D objects from 2D multi-view dental 2D images has been proposed, as shown in Figure 2.6 for some sample data which has been used for that research.



Figure 2.6: Samples From Data Used For Training EfficientNetB0 model for constructing 3D objects from 2D multi-view images [12].

2.2.2 AI tools used to generate CAD models

Several tools have been developed for generating CAD models from various types of input, such as text or images, as has been previously shown in Table 2.1. In this section, these tools will be briefly explained while being compared with the tool called "Text2CAD," developed in [10]. This tool will be discussed in detail in the next chapter, but briefly, it is a two-stage model, where the first stage is an annotation pipeline that transforms a matrix of 2D photos into a text description, which is at an expert level similar to a CAD sequence, and the second stage transforms the generated text by the first stage into a structured CAD sequence and a CAD model in STEP format. It should be mentioned that the annotation pipeline has not been published by the authors of the Text2CAD tool; as a consequence, the Qwen-2.5 VLM model has been fine-tuned to generate the desired non-structured CAD sequence. That will be discussed in the next chapter. Also, the generation of the CAD sequence was not a direct output from the published model, but rather a result of the modified Text2CAD code.

In this context, GenCAD [2], a generative model that adopts autoregressive transformers and latent diffusion models to transform image inputs into

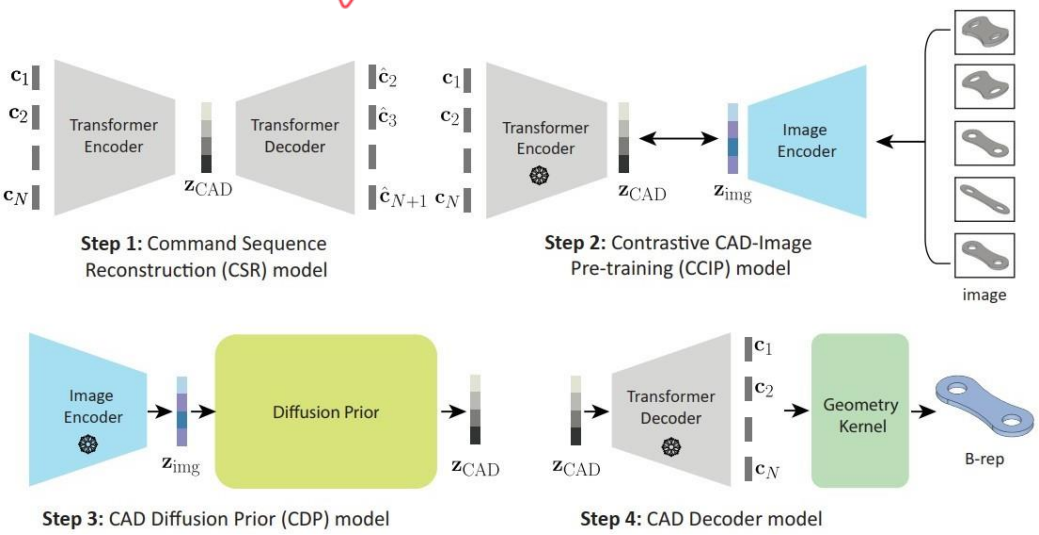


Figure 2.7: GenCAD: The proposed framework, GenCAD, consists of four steps: 1) representation learning of CAD programs or command sequences, 2) learning multimodal latent space of CAD command sequences and CAD images, 3) latent diffusion prior–image conditional CAD generation can be achieved by sampling CAD latents from the diffusion model conditioned on image latents, and 4) decoding CAD latents to CAD command sequences [2].

parametric CAD command sequences, results in editable 3D shape representations, as shown in Figure 2.7. However, the final output of GenCAD is not merely a 3D solid model but a CAD program, which is an entire sequence of parameterized CAD commands. This CAD program can be converted to any CAD form using any off-the-shelf geometry kernel. In comparison with Text2CAD, other than providing a CAD sequence, it provides a ready-to-use STEP file for the CAD model. Also, the code for Text2CAD is published on GitHub, and a tremendous amount of help is provided by the authors for any user who faces problems, which is in contrast with GenCAD.

The data which have been generated using GenCAD [2], have been used to develop VLM tool which generate editable CAD code (CadQuery Python) directly from visual input, this tool called "CAD-CODER" [6], an overview of this tool is shown in Figure 2.8. It has been argued that CAD-CODER outperform widely used state-of-the-art VLM baselines such as GPT-4.5 and Qwen2.5-VL-72B. However, at the start of the current project, CAD-CODER code has not been published.

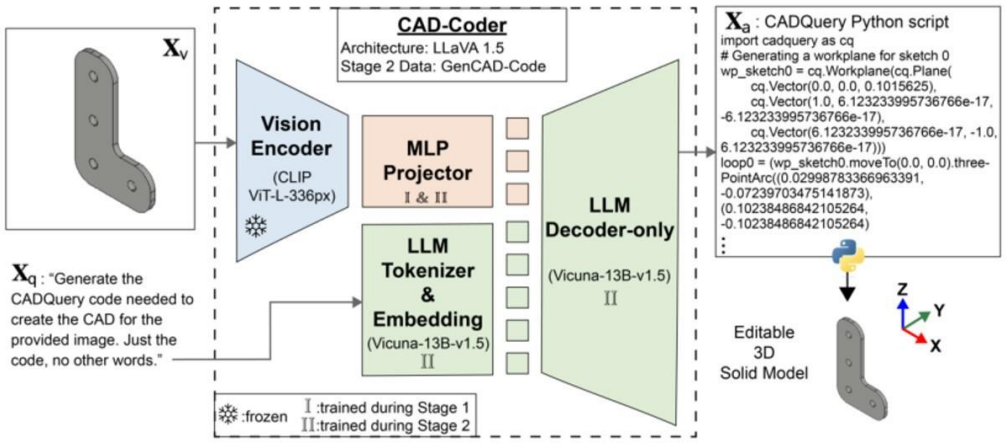


Figure 2.8: Overview of CAD-CODER: The VLM accepts an image as input and outputs CADQuery code [6].

Furthermore, in [13] a Vision Language Model (VLM)-based Multi Agent System(MAS) has been developed which mirrors industry product development team structure which usually involves teams of engineers, requirements engineers, CAD itself, and quality assurance, as shown in Figure 2.9, and an example of the workflow of this model is shown in Figure 2.10, but unfortunately, the code of this model has not been shared yet.

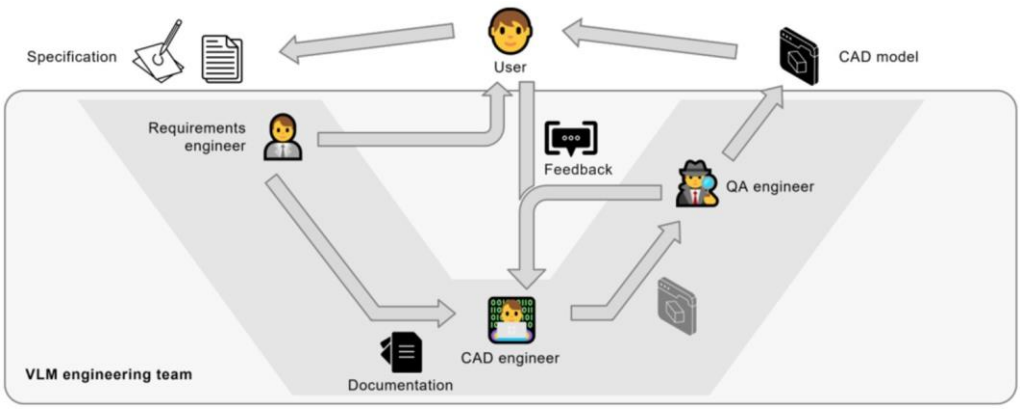


Figure 2.9: Engineering team architecture [13].

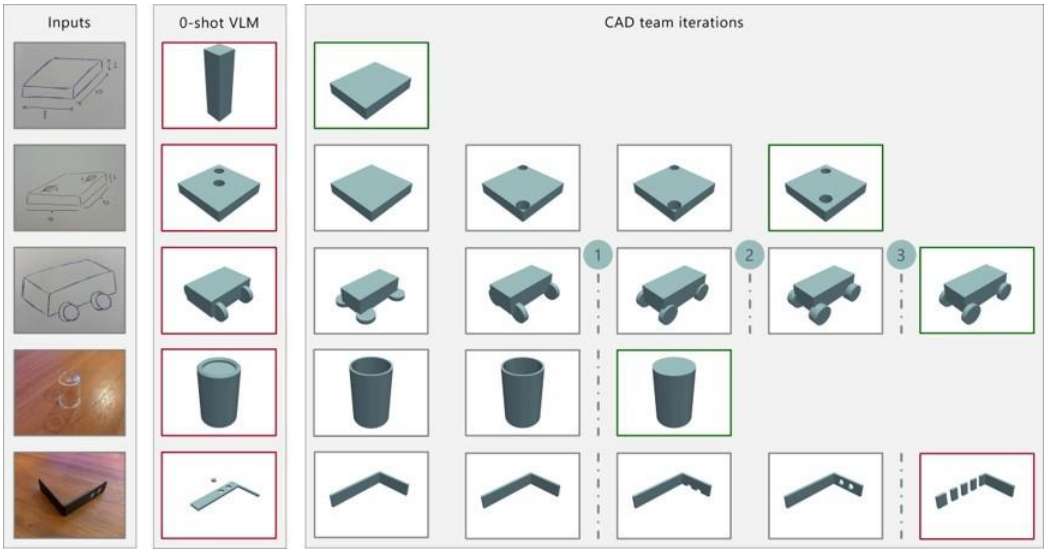


Figure 2.10: Examples of the iterative design process from the user specification to the resulting model. The first column "Inputs" shows the visual inputs, the second column "0-shot VLM" shows the baseline results without requirement clarification, verification and validation, and the right part shows the iterations with the Multi Agent System. The dashed lines indicate validation steps. Red borders indicate wrong CAD models, green borders indicate acceptable designs [13].

Conclusion, Summary of the Lit. work?

Chapter 3

Implementation

In this chapter, the implementation of the previously shown approach will be clarified and discussed in detail.

3.1 STL format to matrix of photos

The first step as discussed in the research plan Figure 2.3 is concerned by generating 2D views of the CAD object in a matrix format. The rotation matrices about the principal axes are defined in Equations (3.1)–(3.3), and the overall rotation matrix can then be expressed as the product of these three matrices, as shown in Equation 3.4 [11]. The implementation in Python scripting for FreeCAD and the resulting matrix for an example are shown in Figure 3.1.

$$P_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\omega_x & \rightarrow \sin\omega_x \\ 0 & \sin\omega_x & \cos\omega_x \end{bmatrix} \tag{3.1}$$

$$P_y = \begin{bmatrix} \cos\omega_y & 0 & \sin\omega_y \\ 0 & 1 & 0 \\ -\sin\omega_y & 0 & \cos\omega_y \end{bmatrix} \tag{3.2}$$

$$P_z = \begin{bmatrix} \cos\omega_z & \rightarrow \sin\omega_z & 0 \\ \sin\omega_z & \cos\omega_z & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{3.3}$$

$$P = P_z P_y P_x$$

$$= \begin{matrix} \cos\omega_z \cos\omega_y & \cos\omega_z \sin\omega_y \sin\omega_x \rightarrow \sin\omega_z \cos\omega_x & \cos\omega_z \sin\omega_y \cos\omega_x + \sin\omega_z \sin\omega_x \\ \sin\omega_z \cos\omega_y & \sin\omega_z \sin\omega_y \sin\omega_x + \cos\omega_z \cos\omega_x & \sin\omega_z \sin\omega_y \cos\omega_x \rightarrow \cos\omega_z \sin\omega_x \\ -\sin\omega_y & \cos\omega_y \sin\omega_x & \cos\omega_y \cos\omega_x \end{matrix} \tag{3.4}$$

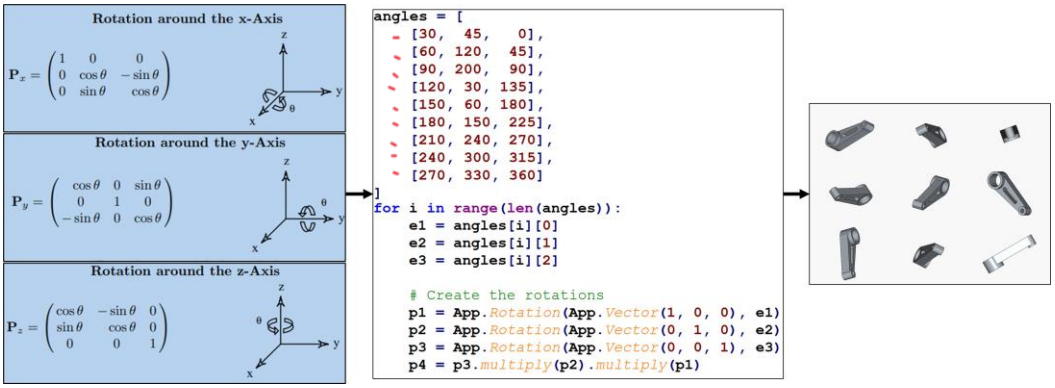


Figure 3.1: Use the rotation matrices along the x, y, and z axes to generate the 3×3 views [11].

3.2 Matrix of photos to text description

3.2.1 Qwen 2.5-Model (VLM)

The Qwen 2.5-VL [3] integrates a vision encoder and a language model decoder to process images and videos as shown in Figure 3.2. This model introduces dynamic resolution processing and absolute time encoding, enabling it to process images of varying sizes and videos of extended durations (up to hours) with second-level event localization.

3.2.2 Data Preparation

The data[1]generated by the Text2CAD research work has been used to fine-tune the Qwen-VLM. This data consists of approximately 170K data points, which consist of a 2D matrix of nine views for every CAD model and an expert non-structured CAD sequence. This data has been prepared as shown in Figure 3.3. When using it for fine-tuning the VL model, it must follow a strict

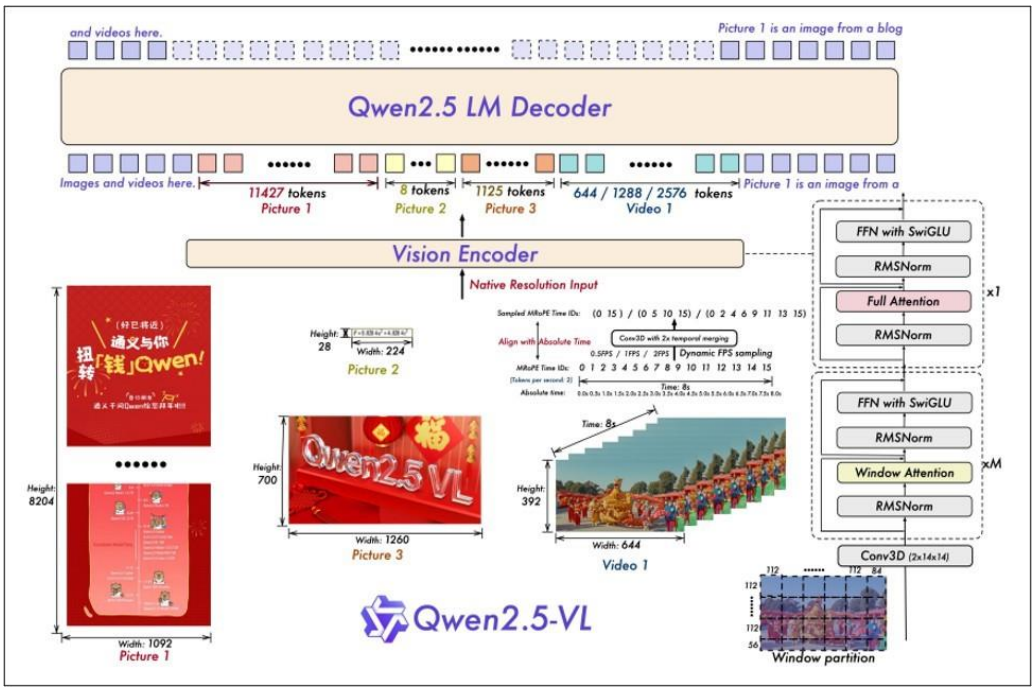


Figure 3.2: The Qwen2.5-VL framework [3].

formulation consisting of three main sections, as illustrated in Figure 3.4: the role, which represents the system; the user, who provides the system with a photo and asks what needs to be extracted from it; and finally, the assistant, which contains the answer to the query. During fine-tuning, the photos are inserted in the user section, while the corresponding answers to the queries are placed in the assistant section.

The system message is a typically used text, and the message used is:

> "You are a highly advanced Vision Language Model (VLM), specialized in analyzing, describing, and interpreting visual data of mechanical objects. Your task is to process and extract meaningful insights from images, leveraging multimodal understanding to provide accurate and contextually relevant descriptions of the data."

For the user section, the query used is very important to be representative of the knowledge which could be extracted from the photo given. In general, every data point should have its corresponding query, but because in the current situation the knowledge required from every photo is the same, this

[1]https://huggingface.co/datasets/SadilKhan/Text2CAD/tree/main

query is the same and it is:

> "Please analyze the attached image showing nine different views of a single mechanical part. Provide a detailed, expert-level description of its geometry and inferred dimensions based on its shape and proportions."

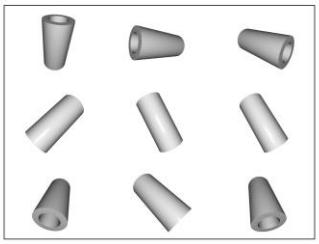
uid	photo	expert
uid 0000/00003166		.. **cylindrical object with a hollow center.** Begin by setting up a new coordinate system with Euler angles of **(0.0, 0.0, -90.0)** and a translation vector **of (0.0, 0.75, 0.0).** .. create a sketch .. consists of two loops, loop 1 and loop 2, on a .. **Loop 1** is a **circle** with a **center** at **(0.192, 0.192)** and a **radius of 0.192.** .. **Loop 2** is also a **circle** with the same **center** at **(0.192, 0.192)** with a **radius of 0.12.** .. sketch **scale parameter of 0.384** and transform it into a 3D sketch. .. create **an extrusion along the normal direction** by a depth **of 0.75,**

Figure 3.3: Data input in pandas DataFrame with columns “uid”, “photo”, and “expert”.

3.2.3 Low-Rank Adaptation of language models

Fine-tuning of large language models involves tweaking a large number of parameters. In [9], Low-Rank Adaptation (LoRA) has been proposed, which freezes the pretrained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly reducing the number of trainable parameters for downstream tasks. This idea and an example are shown in Figure 3.5. In this system, the LoRA matrices A and B are fine-tuned separately, as the sum of elements of both is very small in comparison with the number of elements of their multiplication. The multiplication step is done only during the calculation of loss, evaluation, and when trying to test the fine-tuned model on data. It is worth mentioning that this matrix decomposition process involves dimensional reduction, which means information loss; however, it has been argued that the VLM model itself does not need this whole large set of parameters, and a reduced number of parameters is enough.

```
def format_data(sample):
    return [
        {
            "role": "system",
            "content": [{"type": "text", "text": system_message}]
        },
            "role": "user",
            "content": [
                {
                    "type": "image",
                    "image": sample["photo"],
                    "uid":   sample["uid"],
                },
                {
                    "type": "text",
                    "text": query,
          {
              "role": "assistant",
              "content": [{"type": "text", "text": sample["expert"]}
          },
```

Figure 3.4: The format used to prepare the data for fine-tuning the Qwen VL model.



Figure 3.5: Clarification of Low-Rank Adaptation and an example [9].

What have been done?

3.2.4 Fine-tuning of the Qwen VLM

Fine-tuning procedure

Fine-tuning of VLMs/LLMs follows different frameworks based on the size of the data provided, the structure of the data, and the frequency of updates of the data. There are several techniques to fine-tune language models; some of these techniques could be summarized but not limited to the sequential and the parallel schema, as shown in Figures 3.6. The sequential schema is based on fine-tuning a LoRA layer on a data portion and then keeping fine-tuning it on the subsequent data portions. This technique is based on the assumption that the data is homogeneous, and given that the data used in this project is homogeneous, this technique will be used. The other technique is based on parallel fine-tuning of LoRA layers; in this technique, several LoRA layers will be fine-tuned on different data portions and then combined in the fine-tuned model by weights for each layer, for example: w1, w2, as shown in Figure **??**. These ratios are meant to represent every LoRA layer that has been fine-tuned on a corresponding dataset, which should have a specific construction different from others—for example, a dataset that contains cylindrical objects, another dataset that contains square-based objects, and so on.

Analysis of Fine-tuning results

The Qwen 2.5-Model has been fine-tuned on around 170K data points, which have been prepared as shown in Figures 3.3 and 3.4, based on the sequential schema as shown in Figure 3.6, because the data is homogeneous. The data has been devided into six portions, and the six fine-tuning executions have been caried out, the details of each execution is shown in Table 3.1. These executions have been done on one GPU node 1-NVIDIA L40[2]. The homogeneity of the data has been shown numerically as illustrated in Figure 3.7, where every LoRA after every run is evaluated on the six datasets. The pre-trained LoRA evaluated on the whole datasets is the "Pretrained" model, which shows high evaluation loss compared with the subsequent models (R1–R6). Although the LoRA layer "Model-R1" has been fine-tuned only on dataset-1, it also showed good results when it was evaluated on the other datasets. This also explains the loss–accuracy behavior during the six fine-tuning execution processes shown in Figure 3.8. Most of the fine-tuning has been developed during the first fine-tuning; after that, the changes in loss/accuracy are minimal. However, it is believed that the VLM model is learning better about the data structure during (R2–R6), but because there are a lot of numerical values included—for example, several cylinder shapes

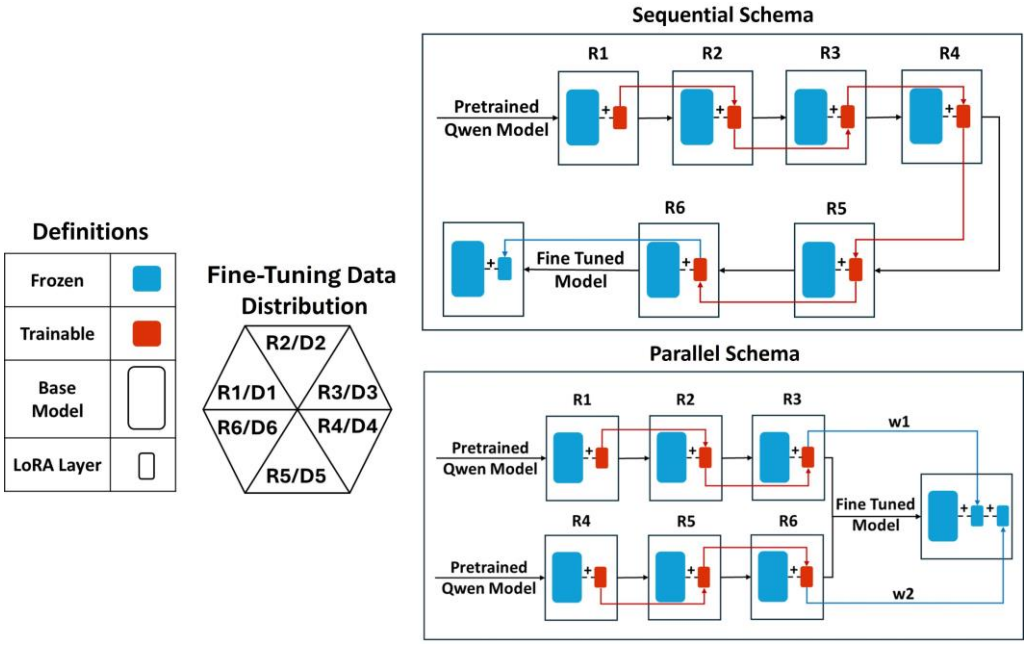


Figure 3.6: The sequential and the paralel scheme to fine-tune Qwen 2.5 VLM.

have different dimensions—and these VL models lack the capability of inferring the numerical measurements, this has also led to the plateau behavior.

[2]Calculations were performed using the festus-cluster of the Bayreuth Centre for High Performance Computing (https://www.bzhpc.uni-bayreuth.de), funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation), and the support from Universität Bayreuth IT-Servicezentrum, René Meißner, is acknowledged

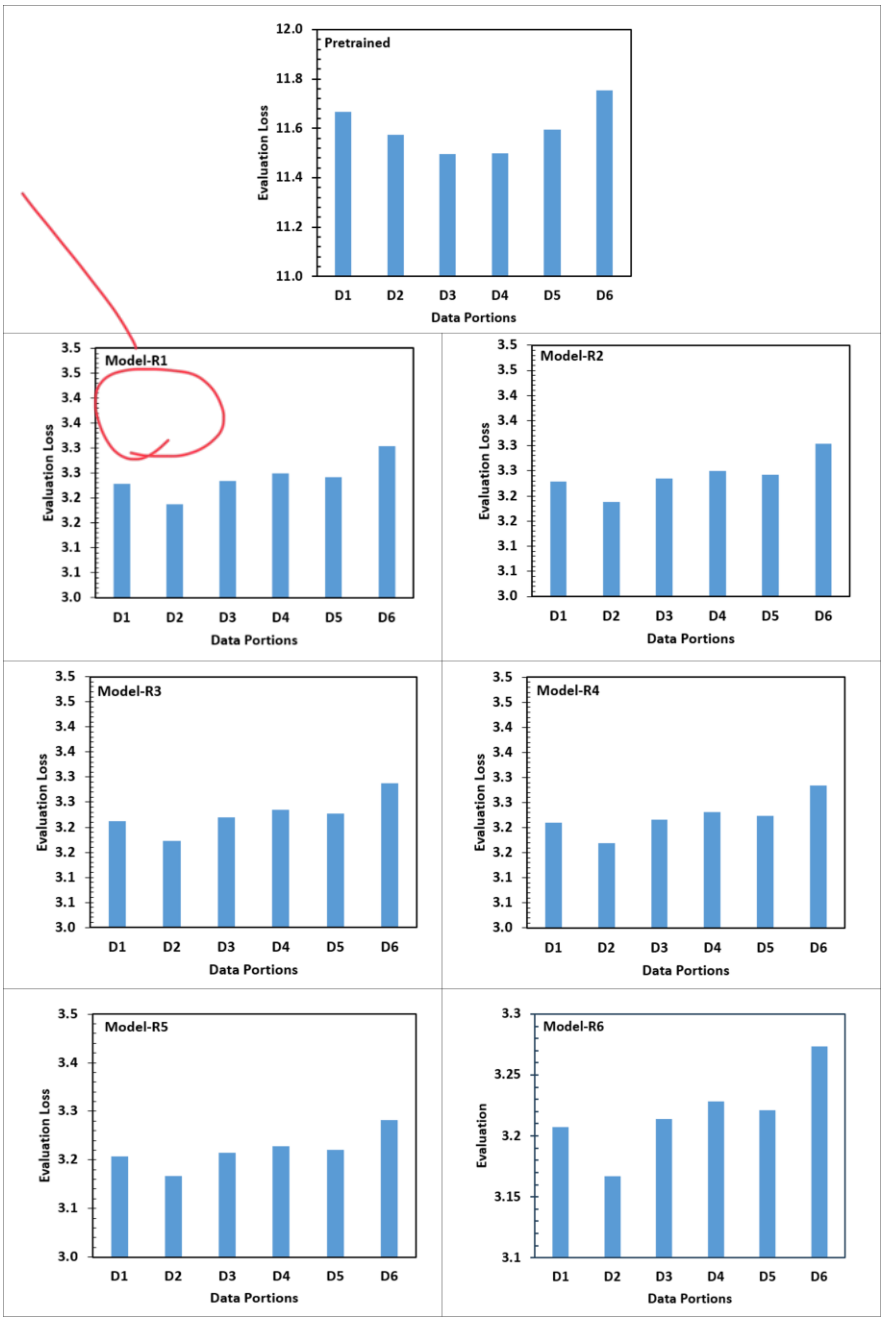


Figure 3.7: The evaluation of each model applied on the whole data sets.

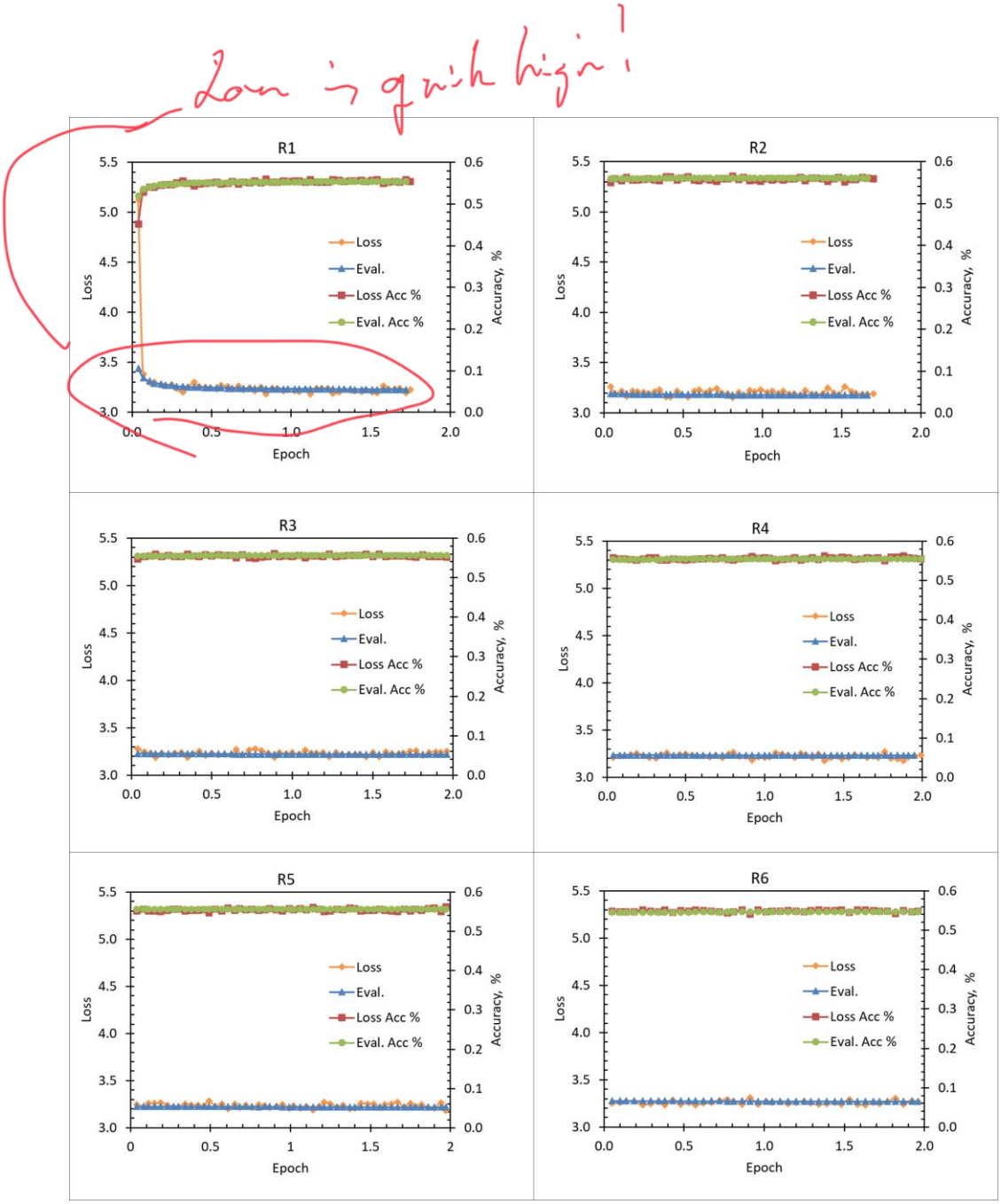


Figure 3.8: The loss, evaluation loss, loss accuracy, and evaluation accuracy.

Table 3.1: Comparison of training configurations and parameters across Runs (R1–R6).

Point of Comp.	R1	R2	R3	R4	R5	R6
Num. of Frozen Param.	8,292,166,656	8,292,166,656	8,292,166,656	8,292,166,656	8,292,166,656	8,292,166,656
Num. of Trainable Param.	2,523,136	2,523,136	2,523,136	2,523,136	2,523,136	2,523,136
All-Param	8,294,689,792	8,294,689,792	8,294,689,792	8,294,689,792	8,294,689,792	8,294,689,792
Data Size	31,711	31,431	28,863	29,012	29,270	23,200
Batch Size / Eval-Steps	2 / 500	2 / 500	2 / 500	2 / 500	2 / 500	2 / 500
Executed Epoch / Time (hrs)	1.72 / 24	1.66 / 24	1.96 / 24	1.99 / 24	1.97 / 24	1.96 / 17

3.3 Text description to CAD sequence

The tool called "Text2CAD," developed in [10], is a two-stage model, where the first stage is an annotation pipeline that transforms a matrix of 2D photos into a non structured CAD sequence, which is at an expert level similar to a CAD sequence, and the second stage transforms the generated text by the first stage into a structured CAD sequence and a CAD model in STEP format, as shown in Figure 3.9. It should be mentioned that the annotation pipeline has not been published by the authors of the Text2CAD tool; as a consequence, the Qwen-2.5 VLM model has been fine-tuned, as it has been discussed in the previous section. Also, the generation of the CAD sequence was not a direct output from the published model, but rather a result of the modified Text2CAD code. This modification[3] is based on exporting the CAD sequence from vector description, as shown in Figure 3.10.

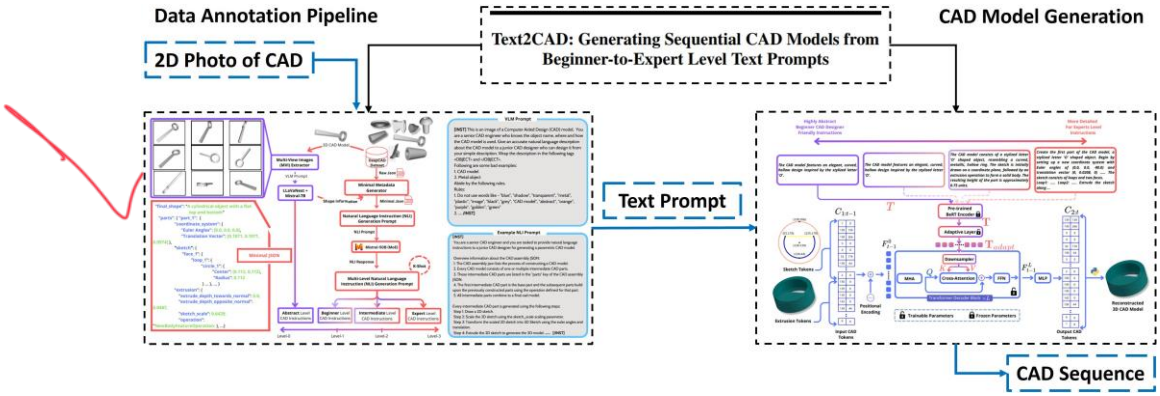


Figure 3.9: The two stages of Text2CAD [10].

[3]Acknowledgment should be given to Mohammad Sadil Khan, the Text2CAD corresponding author, for his help in this implementation.

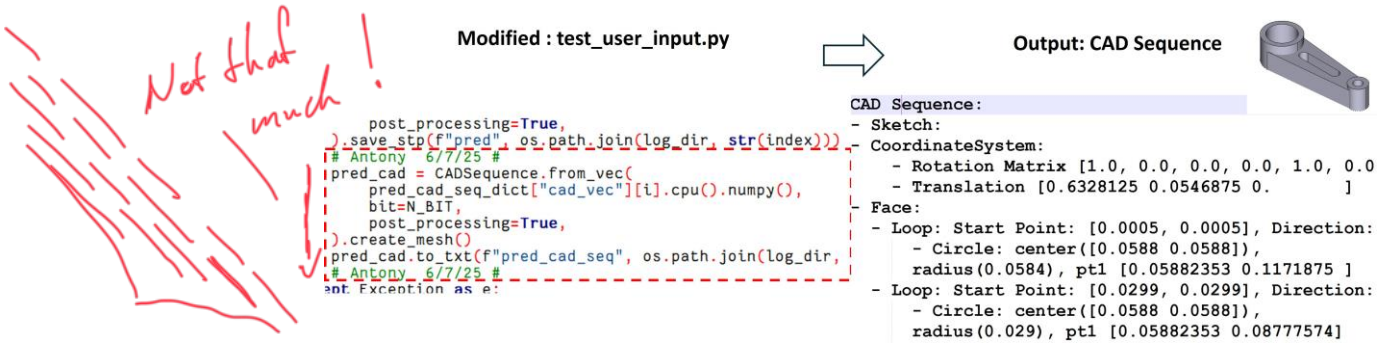


Figure 3.10: Modification for Text2CAD to get structured CAD sequence (Parametric CAD model) as an output.

3.4 CAD sequence to parametric CAD model

In this chapter the previously generated CAD sequence is used to get a parametrized part. CAD sequences contain information in CAD operation style. In order to get a parametrized part this sequence has to be converted into an executable Python script. Every CAD programme has its own Python instructions. For this project Python for FreeCAD is used.

3.4.1 Methodology

Firstly, it has been considered to use an additional LLM to do this conversion. For this, ChatGpt was used initially to convert CAD-sequences into Python script for FreeCAD. However, for complex parts ChatGpt did not produce reliable results. An example of this is shown in Figure 3.11.

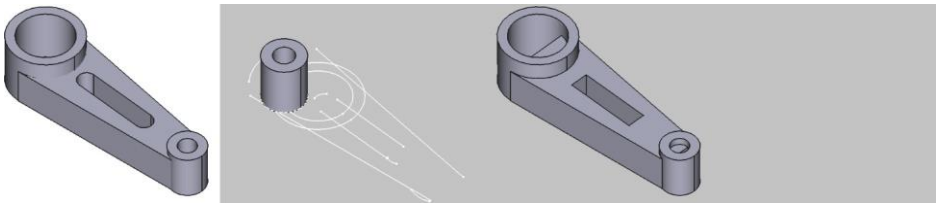


Figure 3.11: Converting CAD-sequence to Python script for FreeCAD using ChatGPT, from left: described part, 3 parts produced by ChatGPT scripts [5].

Moreover, the structure of the generated code varied which can lead to difficulties in the later optimization process. When choosing to fine-tune another LLM to get better results first training data needs to be generated.

Due to the implementation of Text2CAD[10], all CAD sequences follow the same structure. This provides a simpler alternative: A Python script can be used to convert CAD sequences into Python scripts for FreeCAD. This method allows the generation of standardized code making modifications easier to implement.

3.4.2 Implementation

In this section the implementation of the Python script to generate the Python script for FreeCAD is discussed. In this section the focus is on the construction of the geometry. In the next chapter adjustments to this code are made to generate constraints for the optimization process. This changes do not change the geometry.

The input to the script is the text-based CAD sequence which was generated by Text2CAD and follows a fixed structure. It contains two main feature types:

- Sketches: contain relative position of the sketch and 2D geometric elements such as lines, circles and arcs
- Extrusion sequences: define how the sketches are turned into 3D solids

The first processing step after loading the input file is the segmentation into logical blocks. This is done by scanning the input line by line and starting a new block whenever sketch or extrusion headers are found. The blocks are then treated independently based on their type.

Both types of blocks are then parsed into a structured representation which extracts the relevant information in a structured dictionary. This step is done in advance to make the actual code generation easier readable. The generated dictionaries will be used as an input for the actual code generation functions.

In the code generation functions lines are appended to the output script. In the sketch generation function the placement is set using the placement and translation vector which was extracted by the parsing function from the CAD sequence. Then the sketch geometries are generated. For this the generation of lines, circles and arcs with the extracted dimensions are added to the output script. In the extrusion generation function the extrusion length is taken according to the extracted information of the parsing function.

Then the final FreeCAD script is generated. For this firstly, the necessary libraries (FreeCAD, Part, math) are imported. Then the generated code of

the different blocks is added. Multiple bodies are created. In order to have one body all objects are put together into one compound. This is necessary for the optimization process.

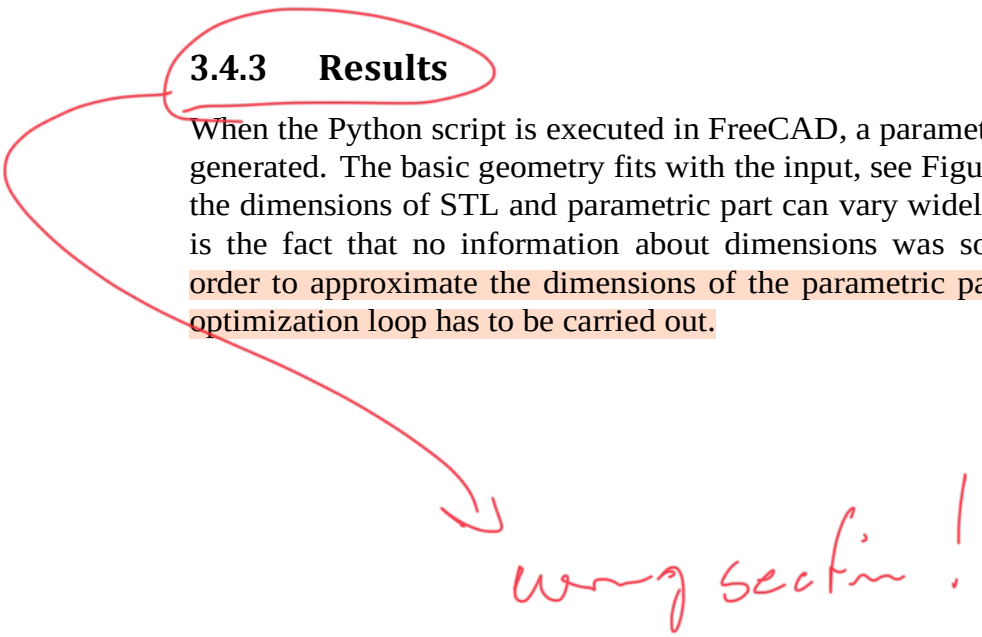
Algorithm 1 shows the pseudocode of the algorithm.

Algorithm 1: CAD sequence to Python for FreeCAD

Input: CAD sequence S
Output: Python script for FreeCAD F
Load CAD sequence text from file S;
Split CAD sequence into ordered blocks (Sketch / Extrude);
Initialize empty FreeCAD script F;
Initialize sketch index $k \uparrow 0$;
foreach *block* B *in sequence* **do**
    if B *is a Sketch block* **then**
        Parse sketch translation, loops, and geometric elements;
        $k \uparrow k + 1$;
        Create FreeCAD sketch object $Sketch_k$;
        Add sketch geometry (lines, circles, arcs);
        Generate geometric and positional constraints;
        (Apply constraints for optimization loop);
        Recompute document;
    else if B *is an Extrude block* **then**
        Parse extrusion parameters (extent, boolean type);
        Create extrusion feature from $Sketch_k$;
        Recompute document;
Create compound object from all extrusion features;
Recompute FreeCAD document;
Write generated FreeCAD script F to file;
return F;

3.4.3 Results

When the Python script is executed in FreeCAD, a parametric CAD model is generated. The basic geometry fits with the input, see Figure 3.12. However, the dimensions of STL and parametric part can vary widely. Reason for this is the fact that no information about dimensions was so far included. In order to approximate the dimensions of the parametric part to the STL, an optimization loop has to be carried out.



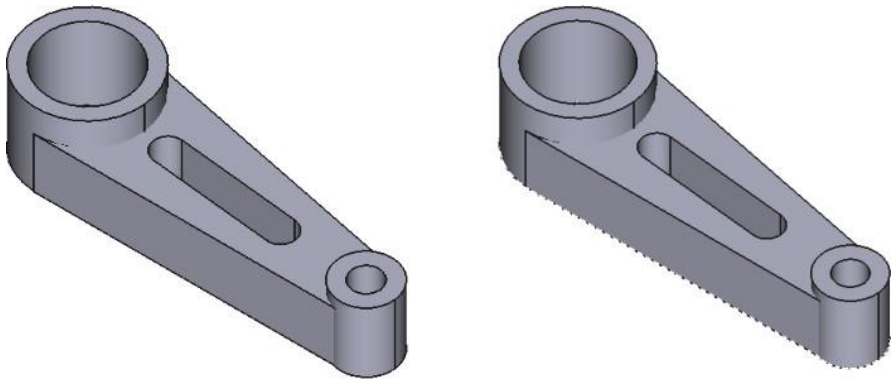


Figure 3.12: Converting CAD-sequence to Python script for FreeCAD using generated Python script, from left: described part, part produced by Python script.

3.5 Optimization process

In this chapter a systematic approach to optimize the alignment of the STL mesh and the parametric CAD model is presented. This is achieved by combining preprocessing, defining a suitable cost function and a robust optimization strategy.

3.5.1 Preprocessing

In order to create a good starting point for the optimization a preprocessing step is performed. Firstly, the STL model is imported into FreeCAD into the same file as the parametric model. The dimensions and alignments of both models initially do not match. To get a reasonable starting point for the optimization algorithm the parametric model is moved such that the bounding boxes have the same origin. Then the parametric model is scaled such that the largest dimensions of the bounding boxes coincide. To avoid distortions this scaling is performed independent of dimension. This step is vital because it removes positioning and scaling effects that could otherwise have dominated the optimization process.

3.5.2 Definition of cost function

In order to implement an optimization method the quality of results must be quantified. For this a cost is defined which defines the deviation of both models. This cost will be minimized by the optimization algorithm.

The cost is defined as the volume of the non-intersecting region between the two models. An example of this is shown in Figure 3.13

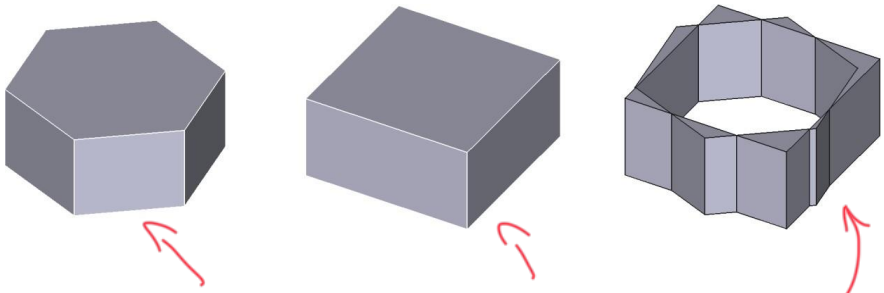


Figure 3.13: left: STL model, middle: parametric model, right: non-intersecting region between the two models.

This can be expressed as:

$$Cost = (V_{parametric} \setminus V_{stl}) + (V_{stl} \setminus V_{parametric})$$

This cost function works for this problem because lower cost implies that the parametric model and the STL match better in terms of space.

3.5.3 Constraint enforcement

Before starting the actual optimization process challenges in parametric optimization are addressed. When optimizing geometry by freely adjusting parameters invalid or degenerated geometries can potentially be generated. For instance coincident lines might be disconnected, features might overlap or the combination of sketch constraints becomes invalid.

To prevent such failures, the previously generated Python script that generates the parametric CAD geometry is modified. Constraints are added ensuring that all sketches are fully determined. For this the script aims to add constraints as long as the sketch is not fully-determined. Firstly, coincident constraints are added to ensure no sketch profiles get disconnected and to keep the sketch extrudable. Secondly, dimensions like lengths or radii, then angle conditions and lastly, positional constraints are added. This order aims to avoid invalid parameter combinations by avoiding having too many positional constraints. The generated constraints are critical because they prevent the optimization process from destroying the sketch-design.

3.5.4 Optimization method

For the actual optimization some suitable algorithm has to be chosen. For CAD-optimization a global optimization over continuous space takes place. An analytical solution to the problem does not exist. Noisy STL-data further restricts the choice of methods. Large search spaces can prevent an exhaustive search. Optimization methods for such problems must be able to locate optima even though only a small part of the space can be sampled. The algorithm should converge to optima while preserving the ability to inspect regions of the search space. Differential evolutionary algorithms fulfill those requirements. [4] Starting from an initial population new candidate solutions are generated by using information of randomly selected individuals within the previous population. For creating the candidates operations like mutation, crossover and selection are used. Candidates are compared to the previous generation and best candidates are used for the new generation. [14]

Algorithm 2 shows the pseudocode of the used evolutionary algorithm.

Algorithm 2: Evolutionary Algorithm

Input: Cost function $f(x)$, initial solution x_0, bounds $[l, u]$, population size N, number of generations G, crossover probability p_c, mutation probability p_m

Output: Best solution $\vec{x}$ (single-objective) or Pareto set P

Initialize population P_0 uniformly within bounds $[l, u]$;
Optionally insert initial solution x_0 into P_0;
Evaluate objective values $F_0 = f(P_0)$;
for $g = 1$ **to** G **do**
    Perform fast non-dominated sorting on $P_{g\uparrow 1}$;
    Compute crowding distance for each Pareto front;
    Create mating pool M using binary tournament selection
    Generate offspring population O using;
        Simulated Binary Crossover (probability p_c);
    Polynomial Mutation (probability p_m)
    Evaluate objective values ;
    Combine populations $R = P_{g\uparrow 1} \downarrow O$;
    Select next generation by rank and crowding distance;
Extract first Pareto front P from P_G;
Select representative solution $\vec{x} \leftrightarrow P$;
return $\vec{x}$ *or* P;

A significant impact to the result is given by the defined bounds for parameters. When choosing large parameter ranges sketches might lose their initial form and result in unwanted geometry changes. Too small ranges for parameters lead to only small adaptions in geometry and not sufficient changes. For sketch dimensions a variation of parameters by ±20% led to best results. For extrusion parameters a variation of ±95% led to the best results. Here more variation is possible, since the larger variation does not destroy the geometry.

When running first drafts of the optimization algorithms undesirable behaviour like degenerated sketches or intersections of sketch elements influenced the results. Reason for this is that the optimization algorithm does not check whether a sketch makes sense. It does not know what the geometry looks like, but only changes the dimensions. As a consequence the optimization loop includes results generated by invalid sketches. This can be local minima, for example when holes in a body are moved to its corners or are overlapping. An example of this is shown in Figure 3.14.

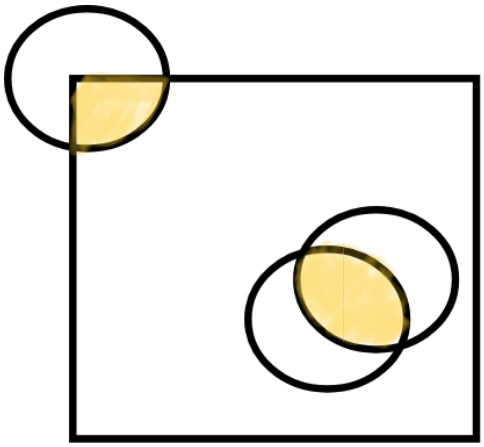


Figure 3.14: Sketch intersections generated by optimization process.

In order to avoid this and to ensure validity and plausibility of the generated models, in that cases penalty terms are added to the cost. By doing this undesirable behaviour is explicitly discouraged. This penalty was set to:

$$intersectionpenalty = numberofintersections \nearrow maximumdimension$$

In Figure 3.15 results are shown for the final optimization algorithm:

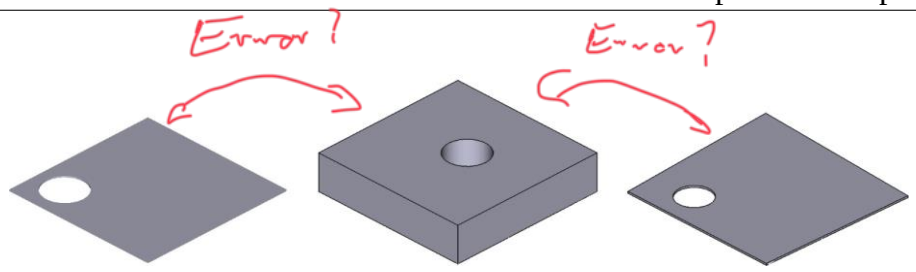


Figure 3.15: Optimization process - left: STL, mid: before optimization, right: after optimization.

The figure shows that the optimization process improves the dimensional fit to the original part significantly. Before optimization the geometries did not match the cost is 24.7, after optimization the cost decreased to 0.33. The evolutionary algorithm is a suitable method for the optimization of CAD geometries. However, when the geometries differ much or small features are contained sometimes the optimization process does not lead to good results. Another factor which affects the quality of results is the number of generations. A large number of generations increases the quality of results but also the time for the optimization. For the previous result 40 generations were chosen. When fast results are needed, this number can be decreased, but will compromise the quality of results.

Chapter 4

Results

In this chapter results are evaluated and failure cases are shown and discussed. First, the results are presented. To evaluate the process, intermediate results from each step are shown and discussed. For this the parts from Figure 4.1 are examined through the workflow.

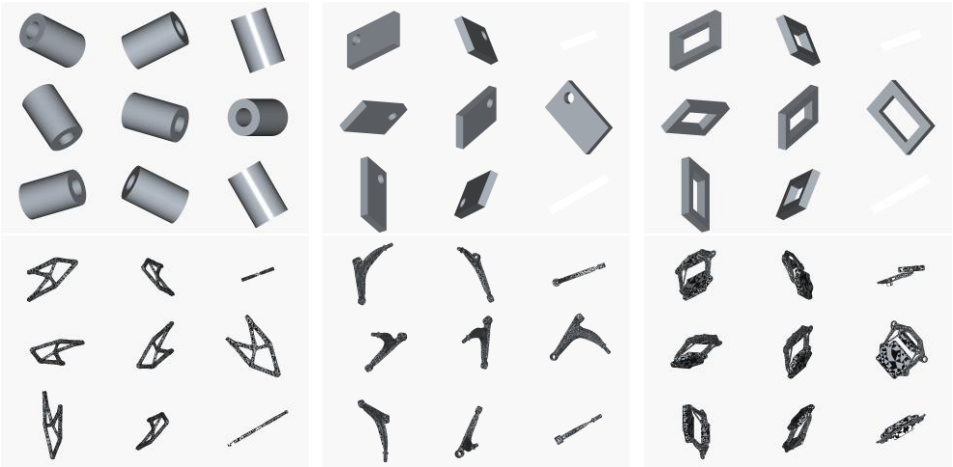


Figure 4.1: Example inputs for the VLM.[1]

The first three parts from Figure 4.1 are simple geometries. Similar geometries were used for training Text2CAD. These parts are considered first. For these examples the LLMs were able to recognize the basic geometry. The initial parts before the optimization process are similar to the STL part. In

[1]The upper three geometries are similar to Text2CAD geometries, the lower three geometries have been provided by Rafinex

Table 4.1 the results before and after optimization[2] are shown in comparison to the STL input.

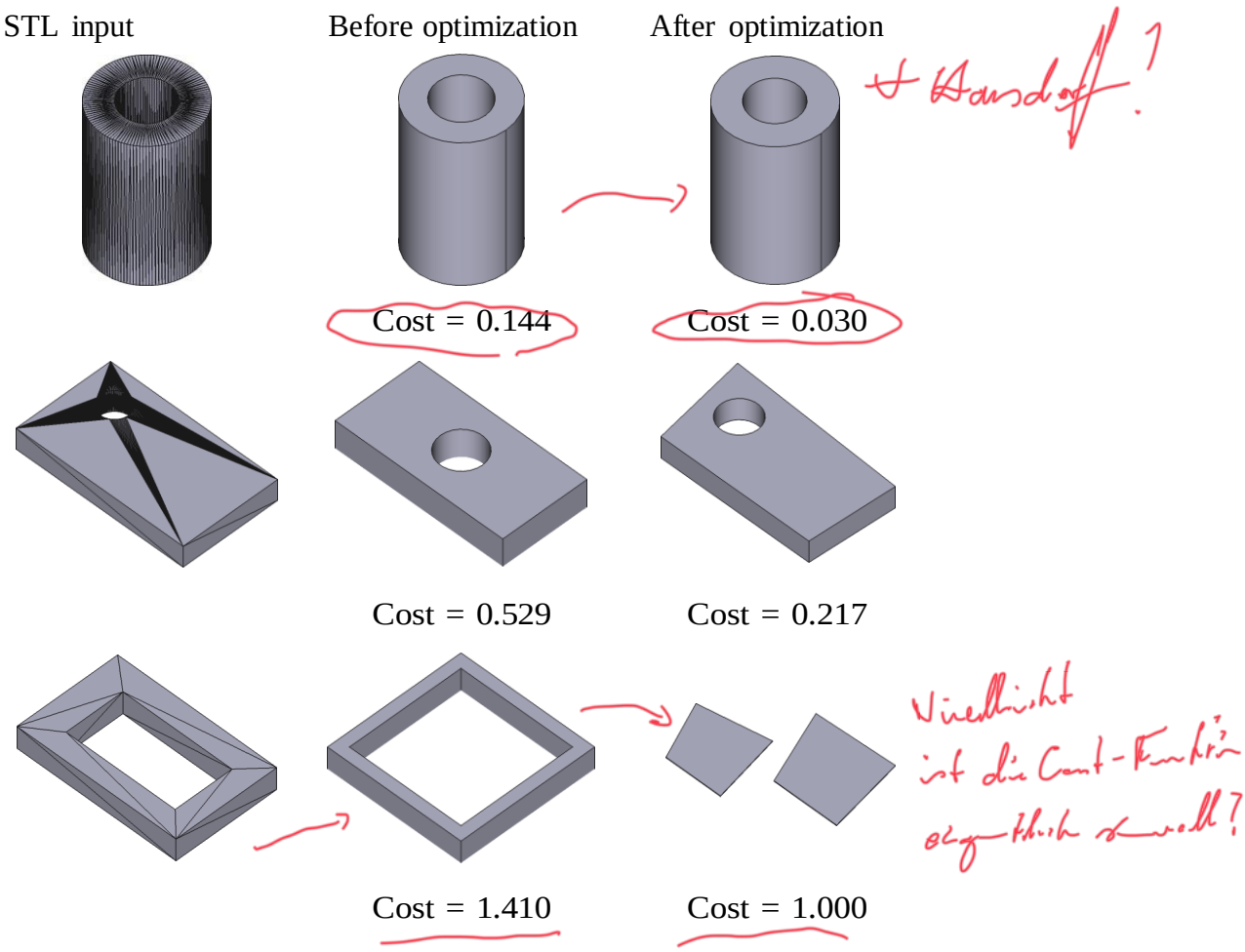


Table 4.1: STL input geometries and corresponding results before and after optimization.

From Table 4.1 it can be seen that the optimization algorithm reduced the cost for all three examples. The result looks fine for the first two examples, but the third geometry deviates from the STL input significantly. Reasons for this are the thin structure of the frame geometry and the small initial overlap of STL input and initial parametric geometry, see Figure 4.2.

The optimization algorithm converged to a local minimum. The extrusion height was reduced significantly, leading to less volume for the parametric

[2] Parameter variation of dimensions by ±20% and of extrusion by ±95% is used

part and a local minimum. The frame structure got lost in this optimization process.

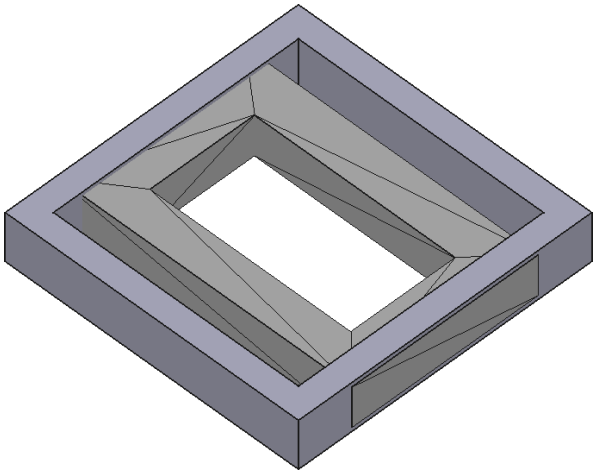


Figure 4.2: Overlap of STL input and initial parametric geometry.

The last three examples from Figure 4.1 show complex geometries which were created using topology optimizations. Now results of these parts will be analyzed. In Table 4.2, the STL input is compared to the geometry before optimization. It can be seen that the geometries differ widely. Reason for this is that complex data was not included in the training data. Since the deviation of the geometry to the STL input is large, an optimization loop will not be carried out because the initial shape is too far away from the input.

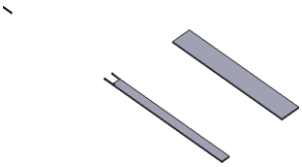
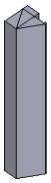
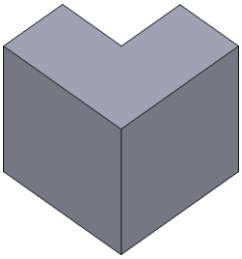
STL input	Before optimization
	 Cost = 1.013
	 Cost = 2.959
	 Cost cannot be computed

Table 4.2: STL input geometries and results before optimization.

Chapter 5

Conclusion and future work

In this project, a method to generate parametric CAD models based on STL meshes has been presented. For this LLMs and optimization techniques were combined. The results were analyzed critically, failure cases and improvement ideas were discussed.

The developed pipeline consists of five steps. Firstly, nine photos from the STL mesh were generated and put into a matrix format. This was then used as input for the visual language model which generated a text description of the photos. Then the generated description was used as an input for Text2CAD. This created a CAD sequence containing the description of the part in form of CAD operations. Using a Python script this was transferred into Python for FreeCAD. When the code is executed in FreeCAD, a parametric part is generated. The basic geometry of the part aligns with the images. Since no information about dimensions was included in the previous steps, the dimensions of STL and parametric part do not match. To address this issue, an optimization loop was implemented to align the parametric model with the STL mesh.

5.1 Future work

As seen previously, the pipeline for generating parametric CAD geometry based on STL input can achieve well matching geometries, but can also fail when small features are included or the geometry is complex. In the following improvement ideas to enhance failure will be discussed.

For thin features the optimization process needs to be modified. It must be ensured that the small features are retained. An idea to improve the result

is chosing a smaller range for optimization parameters, primarily a reduced extrusion scale. The result from this modification is shown in Figure 5.1.

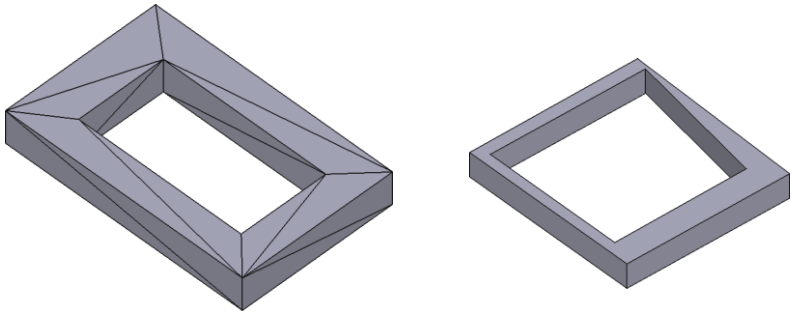


Figure 5.1: Optimization results, from left: STL input, optimization results with parameter variation by ±10% for all parameters.

Also the smaller range of parameters does not always lead to matching geometries. Sometimes the optimization loop gets good results for some parameter combination with low costs, but when applying the optimized parameter combination in the end the cost becomes higher. Reason for this is that there are different valid geometries for the same parameter combination. Even though the sketch is fully constrained, distances can flip there directions unintentionally. An example of this is shown in Figure 5.1.

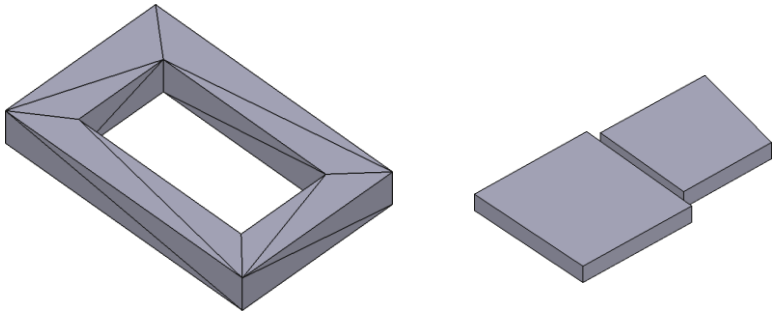


Figure 5.2: Flipped directions in optimization process, from left: STL input, optimization results with parameter variation by ±10% for all parameters.

To guarantee matching results without having to run the optimization algorithm multiple times, the algorithm has to retain directions.

Also the results for complex geometries did not fit to the provided input. Here the result before the optimization differed significantly from the input.

Reason for this is that the LLMs were fine-tuned on more basic geometries and has problems to generalize to complex parts. In order to improve this Qwen 2.5-VLM and Text2CAD have to be fine-tuned with more complex geometry. Additionally, for complex parts more images should be added to the matrix of photos. Also images of cross-sections might improve feature recognition for complex parts.

In order to optimize the results, Qwen 2.5-VLM and Text2CAD should be fine-tuned with more complex data and the optimization algorithm should retain directions to avoid flipped directions.

Bibliography

[1] Aminu Abdullahi and Kathryn Timonera: *Large Language Model: A Guide To The Question 'What Is An LLM'*. Accessed: 2025-10-16. (2024). URL: https://www.eweek.com/artificial-intelligence/large-language-model/.

[2] Md Ferdous Alam and Faez Ahmed: GenCAD: Image-Conditioned Computer-Aided Design Generation with Transformer-Based Contrastive Representation and Diffusion Priors. In: *arXiv preprint* arXiv:2409.16294 (2024). Accessed: 2025-10-16. DOI: 10.48550/arXiv.2409.16294. URL: https://arxiv.org/abs/2409.16294.

[3] Shuai Bai et al.: Qwen2.5-VL Technical Report. In: *arXiv preprint* arXiv:2502.13923 (2025). Accessed: 2025-10-16. DOI: 10.48550/arXiv.2502.13923. URL: https://arxiv.org/abs/2502.13923.

[4] H. M. Cartwright: An Introduction to Evolutionary Computation and Evolutionary Algorithms. In: *Applications of Evolutionary Computation in Chemistry*. Ed. by R. L. Johnston. Vol. 110. Structure and Bonding. Berlin, Heidelberg: Springer, (2004), pp. 3–36. DOI: 10.1007/b13931. URL: https://doi.org/10.1007/b13931.

[5] OpenAI ChatGPT: *ChatGPT — Python code for CAD sequence*. https://chatgpt.com/share/68f9e984-5740-800b-b3f7-0edb591b547d. Accessed: 2025-10-23. (2025).

[6] Anna C. Doris et al.: *CAD-Coder: An Open-Source Vision-Language Model for Computer-Aided Design Code Generation*. (2025). DOI: 10.48550/arXiv.2505.14646. eprint: arXiv:2505.14646. URL: https://arxiv.org/abs/2505.14646.

[7] Anna C. Doris et al.: CAD-Coder: An Open-Source Vision-Language Model for Computer-Aided Design Code Generation. In: *arXiv preprint* arXiv:2505.14646 (2025). Preprint. Accessed: 2025-10-16. DOI: 10.48550/arXiv.2505.14646. URL: https://arxiv.org/abs/2505.14646.

[8] Kostas Giannis et al.: Predicting 3D particles shapes based on 2D images by using convolutional neural network. In: *Powder Technology*

432 (2024), p. 119122. DOI: 10.1016/j.powtec.2023.119122. URL: https://doi.org/10.1016/j.powtec.2023.119122.

[9] Edward J. Hu et al.: LoRA: Low-Rank Adaptation of Large Language Models. In: *arXiv preprint* arXiv:2106.09685 (2021). Accessed: 2025-10-16. DOI: 10.48550/arXiv.2106.09685. URL: https://arxiv.org/abs/2106.09685.

[10] Mohammad Sadil Khan et al.: Text2CAD: Generating Sequential CAD Models from Beginner-to-Expert Level Text Prompts. In: *arXiv preprint* arXiv:2409.17106 (2024). Accepted in NeurIPS 2024 (Spotlight). Accessed: 2025-10-16. DOI: 10.48550/arXiv.2409.17106. URL: https://arxiv.org/abs/2409.17106.

[11] Carl D. Meyer: *Matrix Analysis and Applied Linear Algebra*. SIAM, (2001). URL: http://www.matrixanalysis.com/.

[12] Waleed Mohamed et al.: 3D reconstruction from 2D multi-view dental 2D images based on EfficientNetB0 model. In: *Scientific Reports* 15.1 (2025). DOI: 10.1038/s41598-025-12861-3. URL: https://doi.org/10.1038/s41598-025-12861-3.

[13] Felix Ocker et al.: *From Idea to CAD: A Language Model-Driven Multi-Agent System for Collaborative Design*. 11 pages, 3 figures. (2025). DOI: 10.48550/arXiv.2503.04417. eprint: arXiv:2503.04417. URL: https://arxiv.org/abs/2503.04417.

[14] Rainer Storn and Kenneth Price: Differential Evolution—A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces. In: *Journal of Global Optimization* 11.4 (1997), pp. 341–359. DOI: 10.1023/A:1008202821328. URL: https://doi.org/10.1023/A:1008202821328.

[15] M. Szilvsi-Nagy and Gyula Mátyási: Analysis of STL files. In: *Mathematical and Computer Modelling* 38 (2003). DOI: 10.1016/S0895-7177(03)90079-3.

[16] Yang Zhao, Hao Zhang, and Xiuyuan Hu: Role Taxonomy of Units in Deep Neural Networks. In: *Proceedings of UniReps: The First Workshop on Unifying Representations in Neural Models*. Vol. 243. Proceedings of Machine Learning Research. PMLR, (2024), pp. 291–301. URL: https://proceedings.mlr.press/v243/zhao24b.html.